LOGICAL INSTRUCTIONS

Implementation of Logical instructions using 8086/88 Assembly language.

I. Logical Instructions in Assembly Language

Assembly language implements Logical operations using the following instructions:

<table>
<tr><th>Logical Instructions</th><th>Description</th></tr>
<tr><td>NOT</td><td>Provides 1’s complement of given operand

NOT destination</td></tr>
<tr><td>AND</td><td>Provides 1 if both operand bits are 1 else provides 0 as output

AND destination,source
<table>
<tr><th>X</th><th>Y</th><th>X AND Y</th></tr>
<tr><td>0</td><td>0</td><td>0</td></tr>
<tr><td>1</td><td>0</td><td>0</td></tr>
<tr><td>0</td><td>1</td><td>0</td></tr>
<tr><td>1</td><td>1</td><td>1</td></tr>
</table></td></tr>
<tr><td>OR</td><td>Provides 1 if any of operand bits is 1 else provides 0 as output

OR destination,source
<table>
<tr><th>X</th><th>Y</th><th>X OR Y</th></tr>
<tr><td>0</td><td>0</td><td>0</td></tr>
<tr><td>1</td><td>0</td><td>1</td></tr>
<tr><td>0</td><td>1</td><td>1</td></tr>
<tr><td>1</td><td>1</td><td>1</td></tr>
</table></td></tr>
<tr><td>XOR</td><td>Provides 1 if operand bits are not equal else provides 0 as output

XOR destination,source
<table>
<tr><th>X</th><th>Y</th><th>X XOR Y</th></tr>
<tr><td>0</td><td>0</td><td>0</td></tr>
<tr><td>1</td><td>0</td><td>1</td></tr>
<tr><td>0</td><td>1</td><td>1</td></tr>
<tr><td>1</td><td>1</td><td>0</td></tr>
</table></td></tr>
</table>

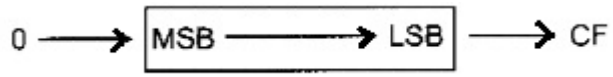
<table>
<tr><th>Logical Instructions</th><th>Description</th></tr>
<tr><td>CMP</td><td>Compare two operands by subtracting the operands and analyzing the carry and zero flags.

CMP destination, source

<table>
<tr><th>Operands</th><th>CF (CARRY FLAG)</th><th>ZF (ZERO FLAG)</th></tr>
<tr><td>Destination > source</td><td>0</td><td>0</td></tr>
<tr><td>Destination = source</td><td>0</td><td>1</td></tr>
<tr><td>Destination < source</td><td>1</td><td>0</td></tr>
</table>
</td></tr>
<tr><td>SHR</td><td>Shift bit of the operand (register or memory location) rightward once, LSB goes to Carry Flag (CF) and MSB is filled with zero

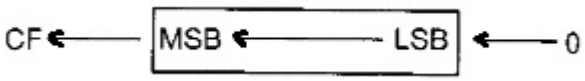
SHR AL, 1

CL can used to hold the count if the bits are required to be shifted more than once

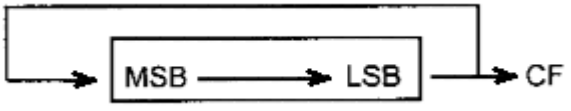
MOV CL, 4
SHR AL, CL
 
</td></tr>
<tr><td>SHL</td><td>Shift bit of the operand (register or memory location) leftward once, MSB goes to Carry Flag (CF) and LSB is filled with zero

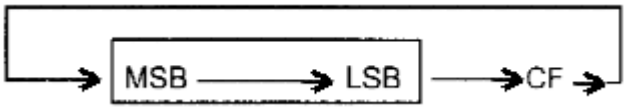
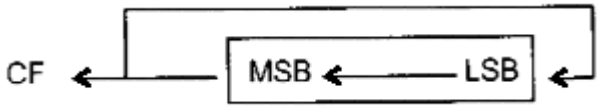
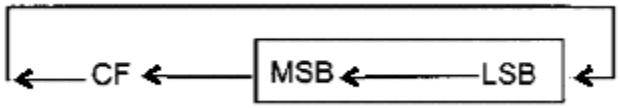
SHL AL, 1

CL can used to hold the count if the bits are required to be shifted more than once

MOV CL, 4
SHL AL, CL
 
</td></tr>
<tr><td>ROR</td><td>Move LSB to MSB once and also copies the LSB to Carry Flag (CF)

ROR AL, 1

 
</td></tr>
</table>

Logical Instructions	Description
RCR	Move LSB to MSB once and also copies the LSB to Carry Flag (CF) **RCR AL, 1** 
ROL	Move LSB to MSB once and also copies the LSB to Carry Flag (CF) **ROL AL, 1** 
RCL	Move LSB to MSB once and also copies the LSB to Carry Flag (CF) **RCL AL, 1** 

Application- 1:

Assume that there is a class of five people certain grades. The highest grade can be computed by comparing the values one by one.

Application- 2:

To convert alphabets of a word or string, e.g., first name of a person, from lowercase to uppercase, a bit-wise AND can be used.

Application- 3:

The number of ones in a byte or word can be computed by shifting the bits and counting if carry flag is 1**.**